

Testing Methodology

Each program was compiled and executed using a C++ compiler. Multiple test inputs were used to confirm correct functionality, pointer handling, dynamic memory allocation, and struct/class access.

Test Case Results

Screenshots were captured during execution to verify correctness. The output matched the expected values in all scenarios. Pointer-based access and memory cleanup behaved correctly.

```
/Users/coltonzachary/.zshrc:23: no such file or directory: /usr/libexec/java_home
● (base) coltonzachary@Coltons-MacBook-Air-4 lab2 % g++ -g lab2-p1.cpp
● (base) coltonzachary@Coltons-MacBook-Air-4 lab2 % ./a.out
87.5
82
● (base) coltonzachary@Coltons-MacBook-Air-4 lab2 % g++ -g lab2-p2.cpp
● (base) coltonzachary@Coltons-MacBook-Air-4 lab2 % ./a.out
Hello structs!
● (base) coltonzachary@Coltons-MacBook-Air-4 lab2 % g++ -g lab2-p3.cpp
● (base) coltonzachary@Coltons-MacBook-Air-4 lab2 % ./a.out

11 21 31 41 51
12 22 32 42 52
13 23 33 43 53
14 24 34 44 54
❖ (base) coltonzachary@Coltons-MacBook-Air-4 lab2 %
```

Observations

The implementation demonstrated correct dynamic memory management, pointer arithmetic, and nested data structure access. No runtime errors or memory leaks were observed during testing.

Conclusion

All lab objectives were successfully completed. The testing confirmed that the implementations function as expected under normal conditions.